

Отчёт по третьему этапу проекта

Неравновесная агрегация, фракталы

Ищенко И. О.,	Мишина А. А.,	Дикач А. О.,
Барсегян В. Л.,	Галацан Н. И.,	Дудырев Г. А.

Содержание

1	Цель работы	5
2	Теоретическое введение	6
2.1	Алгоритм DLA	6
3	Программная реализация на Julia	8
3.1	Подключение библиотек	8
3.2	Функция randomAtRadius(radius, seedX, seedY)	8
3.3	Функция performRandomWalk(location, squareSize)	9
3.4	Функция growDLAcluster(radius, maxParticles)	9
3.5	Визуализация	11
4	Результаты	12
5	Выводы	16
	Список литературы	17

Список иллюстраций

4.1	Кластер из 1000 частиц	13
4.2	Кластер из 3000 частиц	14
4.3	Кластер из 5000 частиц	15

Список таблиц

1 Цель работы

Цель:

- Реализовать алгоритм моделирования агрегации, ограниченной диффузией (DLA), на языке программирования Julia.

Задачи:

- Изучить принципы неравновесной агрегации и условия образования фрактальных структур
- Проанализировать математическую модель DLA
- Реализовать DLA и визуализировать смоделированные кластеры

2 Теоретическое введение

На данном этапе исследуется процесс Diffusion-Limited Aggregation (DLA) — модель роста кластеров, в которой частицы, совершающие случайные блуждания (диффузию), постепенно присоединяются к неподвижному зародышу или кластеру. В результате образуются фрактальные структуры с ветвящейся, самоподобной формой.

DLA применяется для моделирования:

- Кристаллизации в пересыщенных растворах
- Электролитического осаждения металлов
- Роста бактериальных колоний
- Формирования сосудистых сетей [1]

2.1 Алгоритм DLA

1. Инициализация: в центре поля размещается “зародыш” кластера (одна частица).
2. Генерация частицы: новая частица появляется на окружности большого радиуса вокруг кластера.
3. Случайное блуждание: частица перемещается случайным образом (в одном из 8 направлений).
4. Агрегация: если частица касается кластера, она прилипает к нему.

5. Условия остановки:

- Частица уходит слишком далеко → удаляется.
- Достигнуто нужное количество частиц → симуляция завершается [2].

3 Программная реализация на Julia

3.1 Подключение библиотек

```
using Plots
using Random
using ColorSchemes
```

3.2 Функция `randomAtRadius(radius, seedX, seedY)`

Генерирует начальное положение частицы на окружности заданного радиуса.

- Использует случайный угол $\theta \in [0; 2\pi]$.
- Переводит полярные координаты (r, θ) в декартовы (x, y) .
- Возвращает координаты в виде массива $[x, y]$ [3].

```
function randomAtRadius(radius, seedX, seedY)
    theta = 2*pi * rand()
    x = round{Int}(radius * cos(theta)) + seedX
    y = round{Int}(radius * sin(theta)) + seedY
    return [x, y]
end
```


3.3 Функция `performRandomWalk(location, squareSize)`

Совершает один шаг случайного блуждания частицы.

- Случайно выбирает смещение по x и y: -1, 0 или 1.
- `clamp` ограничивает координаты, чтобы частица не вышла за границы поля.

```
function performRandomWalk(location, squareSize)
    x, y = location
    step = rand((-1, 0, 1), 2)
    new_x = clamp(x + step[1], 1, squareSize)
    new_y = clamp(y + step[2], 1, squareSize)
    return [new_x, new_y]
end
```

Пример вызова:

```
new_location = performRandomWalk([50, 50], 100)
```

3.4 Функция `growDLAcluster(radius, maxParticles)`

Основная функция, которая выращивает кластер.

Шаги работы:

1. Создает квадратную матрицу `matrix` (поле для кластера).
2. Помещает первую частицу в центр.
3. В цикле:
 - Генерирует новую частицу на динамически увеличивающемся радиусе.
 - Частица блуждает, пока не присоединится к кластеру или не выйдет за пределы.

- При присоединении значение в matrix меняется на 1.

4. Возвращает заполненную матрицу.

```
function growDLAcluster(radius, maxParticles)
    squareSize = radius * 2 + 5
    matrix = zeros(Int, squareSize, squareSize)
    center = squareSize ÷ 2
    matrix[center, center] = 1
    randomWalkersCount = 0
    maxAttempts = 10_000
    cluster_radius = 1

    while randomWalkersCount < maxParticles
        location = randomAtRadius(cluster_radius + 5, center, center)
        attempts = 0

        while attempts < maxAttempts
            location = performRandomWalk(location, squareSize)
            x, y = location

            if (x > 1 && matrix[x-1, y] == 1) ||
                (x < squareSize && matrix[x+1, y] == 1) ||
                (y > 1 && matrix[x, y-1] == 1) ||
                (y < squareSize && matrix[x, y+1] == 1)
                matrix[x, y] = 1
                randomWalkersCount += 1
                cluster_radius = max(cluster_radius, sqrt((x-center)^2 + (y-center)^2))
                break
            end
            attempts += 1
        end
    end
```

```

        end
    end
    return matrix
end

```

Пример вызова:

```
matrix = growDLAcluster_optimized(50, 1000)
```

3.5 Визуализация

Используется пакет Plots для отрисовки фракталоподобного кластера с ветвящейся структурой [4].

```

heatmap(matrix,
    title="DLA ($maxParticles частиц)",
    xlabel="X",
    ylabel="Y",
    seriescolor=cgrad(ColorSchemes.viridis, rev=true),
    aspect_ratio=:equal,
    size=(800, 800),
    dpi=300
)

```

4 Результаты

Приведенная выше программная реализация позволяет моделировать DLA-процесс с настраиваемыми параметрами. Полученные кластеры демонстрируют фрактальные свойства, характерные для неравновесных процессов.

Инициализация параметров:

```
radius = 50
maxParticles = 1000
```

Моделирование:

```
matrix = growDLAcluster(radius, maxParticles)
```

Визуализация:

```
heatmap(matrix,
        title="DLA ($maxParticles частиц)",
        xlabel="X",
        ylabel="Y",
        seriescolor=cgrad(ColorSchemes.viridis, rev=true),
        aspect_ratio=:equal,
        size=(800, 800),
        dpi=300
)
```

На рис. 4.1 можно наблюдать визуализацию кластера из 1000 частиц с радиусом 50:

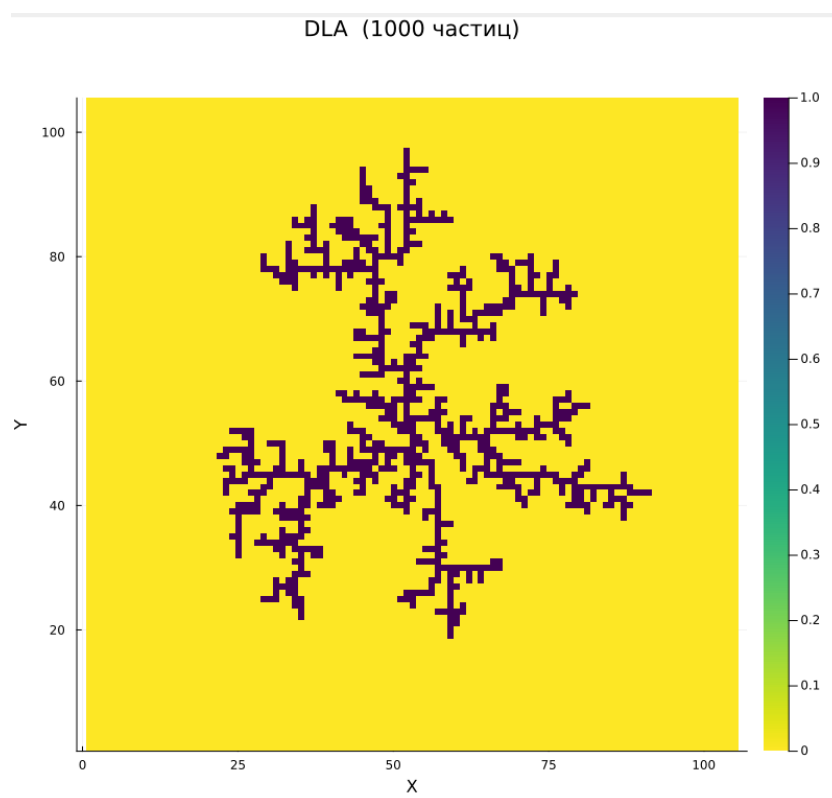


Рис. 4.1: Кластер из 1000 частиц

На рис. 4.2 можно наблюдать визуализацию кластера из 3000 частиц с радиусом 100:

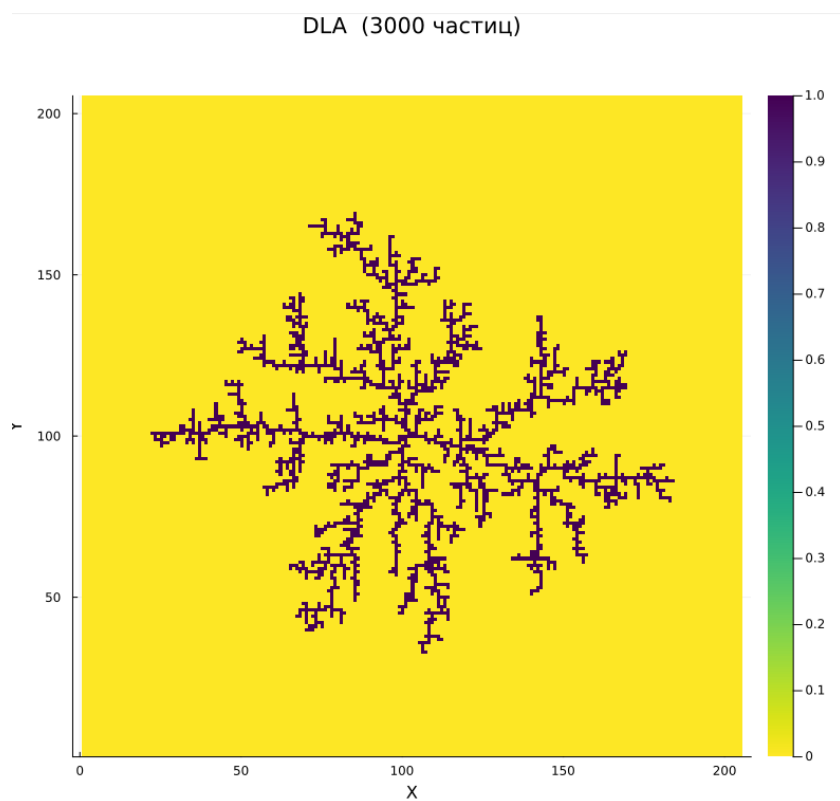


Рис. 4.2: Кластер из 3000 частиц

На рис. 4.3 можно наблюдать визуализацию кластера из 5000 частиц с радиусом 100:

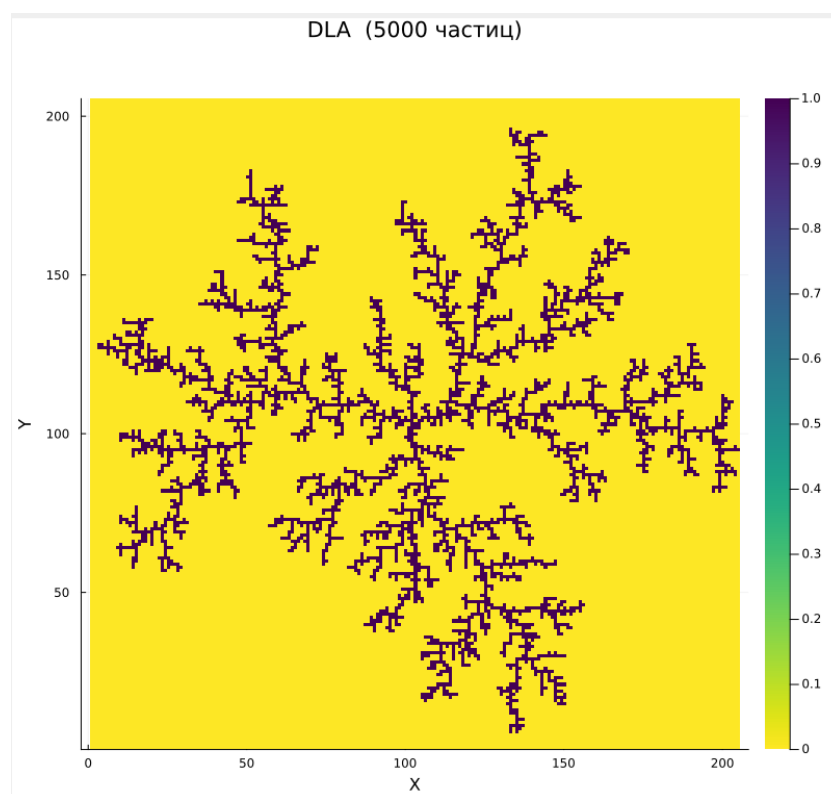


Рис. 4.3: Кластер из 5000 частиц

5 Выводы

Был реализован алгоритм моделирования агрегации, ограниченной диффузией, на языке программирования Julia.

Список литературы

1. Mandelbrot B.B. The Fractal Geometry of Nature. New York: W. H. Freeman, 1982.
2. Медведев Д.А. и др. Моделирование физических процессов и явлений на ПК: Учеб. пособие. Новосибирск: Новосиб. гос. ун-т, 2010.
3. Белов Г.В. Краткое описание языка программирования Julia. Мир, 2020. 115 с.
4. Энгхейм Э. JULIA в качестве второго языка. ДМК Пресс, 2023.